

# **AQI Predictor: Pakistan AQI 3-Day Forecast System**

## **Project Report**

### **1. Project Overview**

This project is an end-to-end machine learning system that predicts the Air Quality Index (AQI) for four major Pakistani cities Islamabad, Karachi, Hyderabad, and Lahore for the next 24, 48, and 72 hours. The system uses a fully serverless architecture: it automatically collects weather and pollution data, engineers' predictive features, trains and compares multiple forecasting models, and serves live predictions through an interactive web dashboard with model explainability.

The goal was to build a complete pipeline covering data engineering, feature engineering, model experimentation, deployment, and monitoring, rather than just a single trained model.

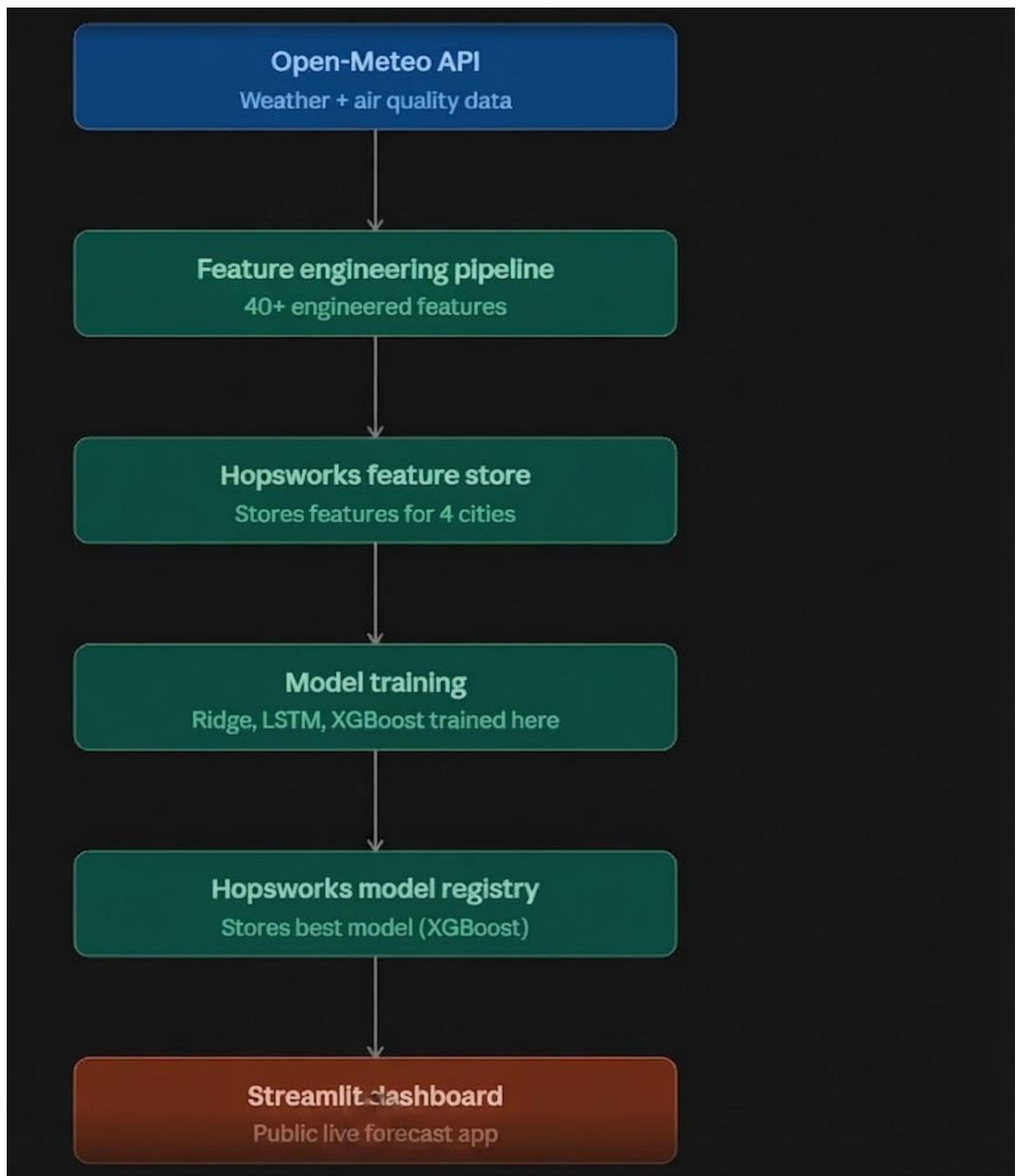
### **2. System Architecture**

The system consists of four main components working together:

- 1. Data Source** – Open-Meteo (free weather and air quality APIs, no API key required)
- 2. Feature Store & Model Registry** – Hopsworks (cloud-hosted, EU-West region)
- 3. Training Environment** – Hopsworks' own Jupyter notebooks
- 4. Dashboard** – Streamlit, deployed publicly via Streamlit Community Cloud

#### **Flow:**

Open-Meteo API → Feature Engineering Script → Hopsworks Feature Store → Model Training (Hopsworks Jupyter) → Hopsworks Model Registry → Streamlit Dashboard → End User



Automation for ongoing data collection was set up through GitHub Actions (hourly feature updates, daily retraining), though these are currently kept as manual-trigger workflows due to a network compatibility issue discussed in Section 6.

### 3. Data Pipeline

#### 3.1 Data Sources

All data is pulled from the free Open-Meteo API:

| Endpoint                                        | Data                                                                                      |
|-------------------------------------------------|-------------------------------------------------------------------------------------------|
| Historical Weather (archive-api.open-meteo.com) | Temperature, humidity, wind speed/direction, precipitation, surface pressure, cloud cover |
| Forecast Weather (api.open-meteo.com)           | Same variables, used for live inference                                                   |
| Air Quality (air-quality-api.open-meteo.com)    | PM10, PM2.5, CO, NO2, SO2, Ozone, US AQI                                                  |

### 3.2 Cities Covered

Islamabad, Karachi, Hyderabad, and Lahore, using their respective latitude/longitude coordinates.

### 3.3 Historical Backfill

Two years of hourly historical data (~15,800 rows per city, ~63,000 rows total) was collected and stored in the Hopsworks Feature Store. To make this process reliable, the following design decisions were made:

- **Monthly chunking** – data inserted in 30-day batches instead of one large insert, to avoid timeouts
- **Resume-safe design** – the script checks what data already exists (by city and timestamp) and skips it, so an interrupted run can be safely restarted
- **Retry logic** – automatic retries with exponential backoff for transient API/network failures
- **Local backup** – every inserted batch is also saved locally as a CSV file, as a safeguard against data loss

## 4. Feature Engineering

From the 14 raw input variables (7 weather + 7 air quality), over 40 features were engineered:

| Category | Count | Description |
|----------|-------|-------------|
|          |       |             |

|                           |    |                                                                                                                 |
|---------------------------|----|-----------------------------------------------------------------------------------------------------------------|
| Time-based                | 5  | Hour, day of week, month, weekend flag, season                                                                  |
| Lag features              | 10 | AQI and PM2.5 values from 1, 3, 6, 24, and 48 hours ago                                                         |
| Rate of change            | 1  | Change in AQI compared to 1 hour ago                                                                            |
| Rolling window statistics | 15 | Min, max, mean, and standard deviation of AQI over 6h, 24h, and 7-day windows; mean PM2.5 over the same windows |
| Raw inputs                | 14 | The original weather and pollution variables                                                                    |
| Prediction targets        | 3  | AQI value 24, 48, and 72 hours into the future                                                                  |

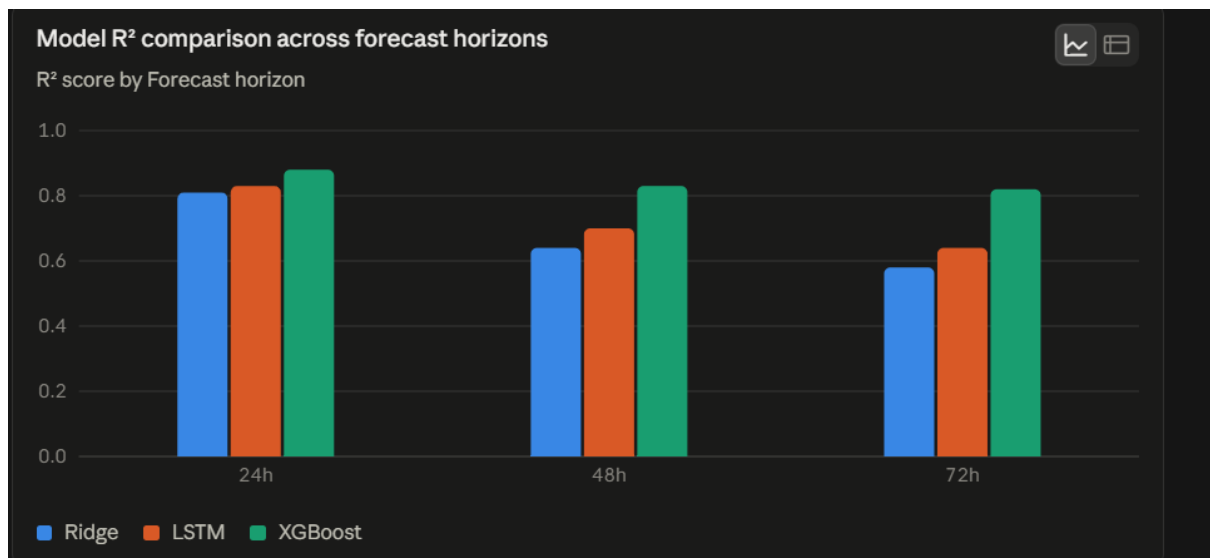
These features were stored with an explicitly defined schema in the Hopsworks Feature Store (48 columns total) to ensure consistency between training and inference.

## 5. Model Training and Evaluation

Three models were trained and compared, covering a range from simple statistical modelling to deep learning, on an 80/20 train-test split (50,688 training rows, 12,672 testing rows):

| Model            | Type                   | Configuration                              |
|------------------|------------------------|--------------------------------------------|
| Ridge Regression | Statistical (baseline) | alpha = 1.0                                |
| LSTM             | Deep learning          | LSTM(64) → Dense(32) → Dense(3), 10 epochs |

|         |                                         |                                                      |
|---------|-----------------------------------------|------------------------------------------------------|
| XGBoost | Gradient boosting<br>(production model) | 100 estimators,<br>learning rate 0.1,<br>max depth 5 |
|---------|-----------------------------------------|------------------------------------------------------|



## Results

| Metric      | Ridge<br>(24h) | LSTM<br>(24h) | XGBoost<br>(24h) | Ridge<br>(48h) | LSTM<br>(48h) | XGBoost<br>(48h) | Ridge<br>(72h) | LSTM<br>(72h) | XGBoost<br>(72h) |
|-------------|----------------|---------------|------------------|----------------|---------------|------------------|----------------|---------------|------------------|
| <b>RMSE</b> | 16.88          | 16.07         | 13.41            | 22.84          | 20.77         | 15.91            | 24.50          | 22.68         | 16.05            |
| <b>MAE</b>  | 11.54          | 11.21         | 9.55             | 16.19          | 14.64         | 11.79            | 17.70          | 16.10         | 12.05            |
| <b>R²</b>   | 0.81           | 0.83          | 0.88             | 0.64           | 0.70          | 0.83             | 0.58           | 0.64          | 0.82             |

XGBoost consistently outperformed both the linear baseline and the LSTM model across all three forecast horizons, and was selected as the production model. It was registered in the Hopsworks Model Registry as `aqi_xgboost_multi`.

## Feature Importance

Using the trained XGBoost model, the most influential features for prediction were:

1. Current AQI (~48% importance)
2. 7-day rolling mean AQI (~12%)
3. 24-hour rolling mean PM2.5 (~7%)
4. 7-day rolling mean PM2.5 (~5%)
5. Current PM2.5 (~2%)

This makes intuitive sense: recent air quality trends are the strongest predictors of near-future air quality.

## 6. Problems Faced and Solutions

Building this system involved solving several real-world engineering challenges:

| Problem                                                                                             | Cause                                                                                         | Solution                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ModuleNotFoundError</code> in GitHub Actions                                                  | Python couldn't resolve the project's internal module structure in the CI environment         | Explicitly set <code>PYTHONPATH</code> in the workflow configuration                                                                                  |
| Timeouts when fetching data from Open-Meteo                                                         | Occasional slow/unresponsive API responses                                                    | Added a 120-second timeout with automatic retry (3 attempts, exponential backoff)                                                                     |
| Hopsworks schema mismatch errors                                                                    | The feature store auto-inferred an inconsistent data schema on first insert                   | Defined the feature schema explicitly in code instead of relying on auto-inference                                                                    |
| Persistent network failures ("HDFS RPC listener disconnected") between GitHub Actions and Hopsworks | A network-level incompatibility between GitHub's cloud runners and Hopsworks' storage backend | Moved the historical backfill and model training to run directly inside Hopsworks' own Jupyter environment, which bypasses this network path entirely |
| Risk of losing backfilled data if the Hopsworks account hit its free-tier usage limit               | No payment method was linked, so continued usage past the free tier could pause the account   | Built a local backup system that saves every inserted batch as a CSV file, allowing quick recovery without needing to re-fetch from the API           |
| SHAP explainability failing with newer XGBoost versions                                             | XGBoost 3.0+ changed how it internally serializes model parameters                            | Patched the affected internal decoding function to correctly parse the new format                                                                     |
| Streamlit Cloud deployment failing to install dependencies                                          | The cloud environment defaulted to a very new Python version (3.14) that had no compatible    | Manually set the deployment's Python version to 3.11 via Streamlit Cloud's settings, and removed the TensorFlow                                       |

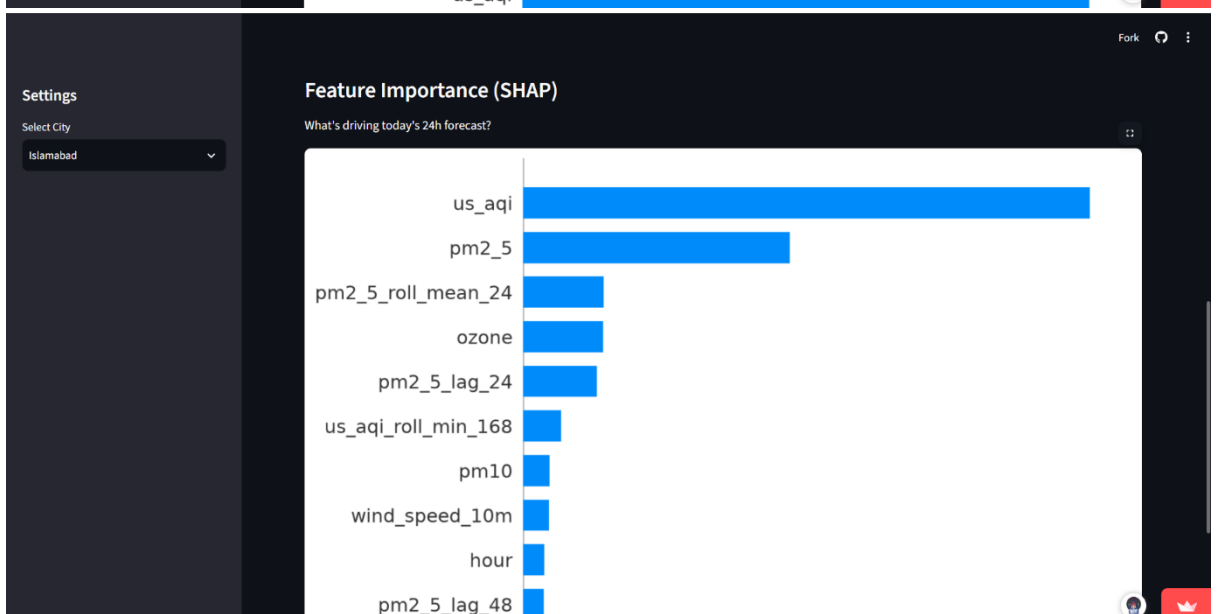
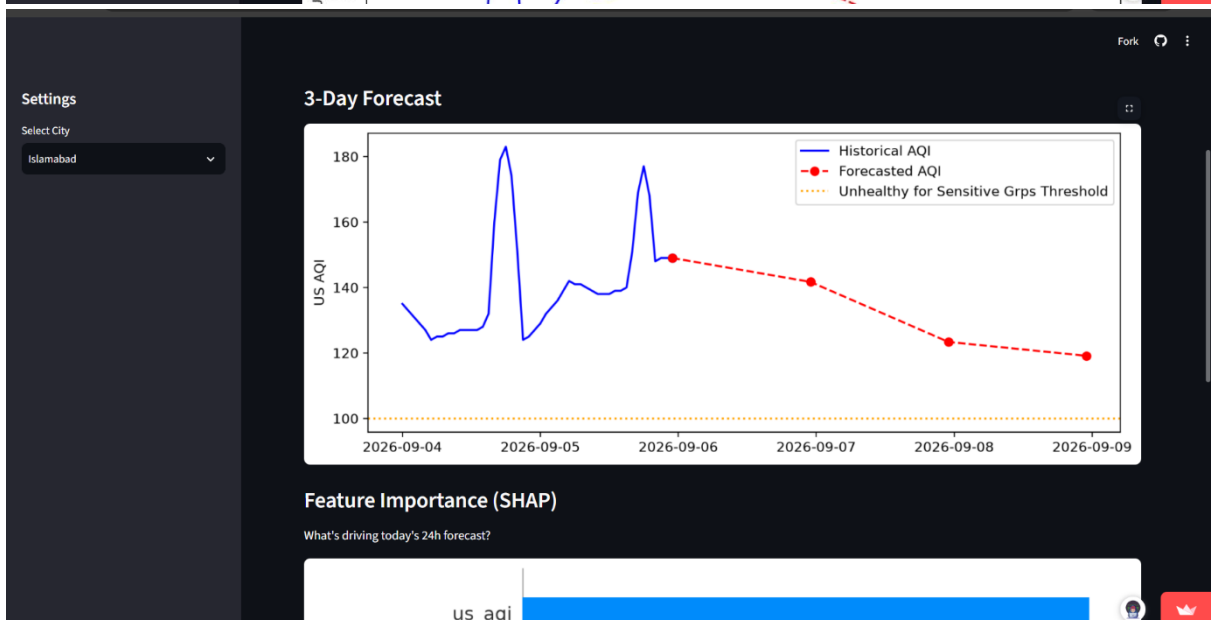
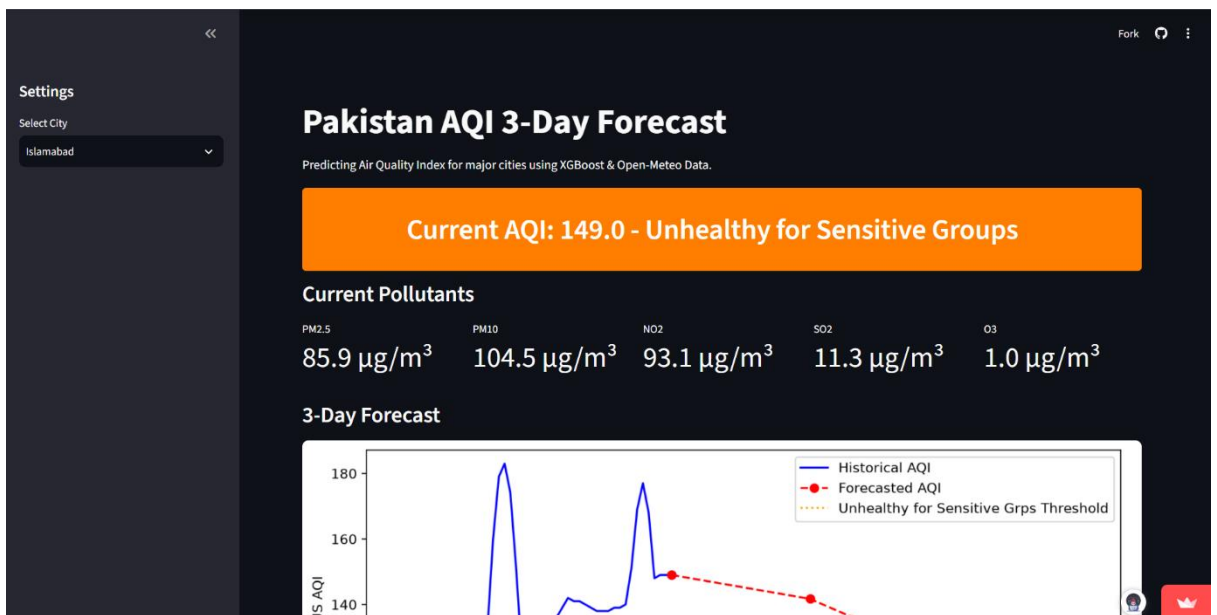
|                                                                               |                                                                                                       |                                                                                                                                                                        |
|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                               | TensorFlow build, and could not compile pandas from source                                            | dependency from the deployed app (since the LSTM model, which needs TensorFlow, is only used for offline comparison — the live dashboard only needs the XGBoost model) |
| A Windows-specific package ( <code>twofish</code> ) failed to install locally | It is a transitive dependency of the Hopsworks package that requires a C compiler to build on Windows | Documented the issue and workaround (installing Microsoft C++ Build Tools, using WSL, or running inside Hopsworks Jupyter) in the project README                       |

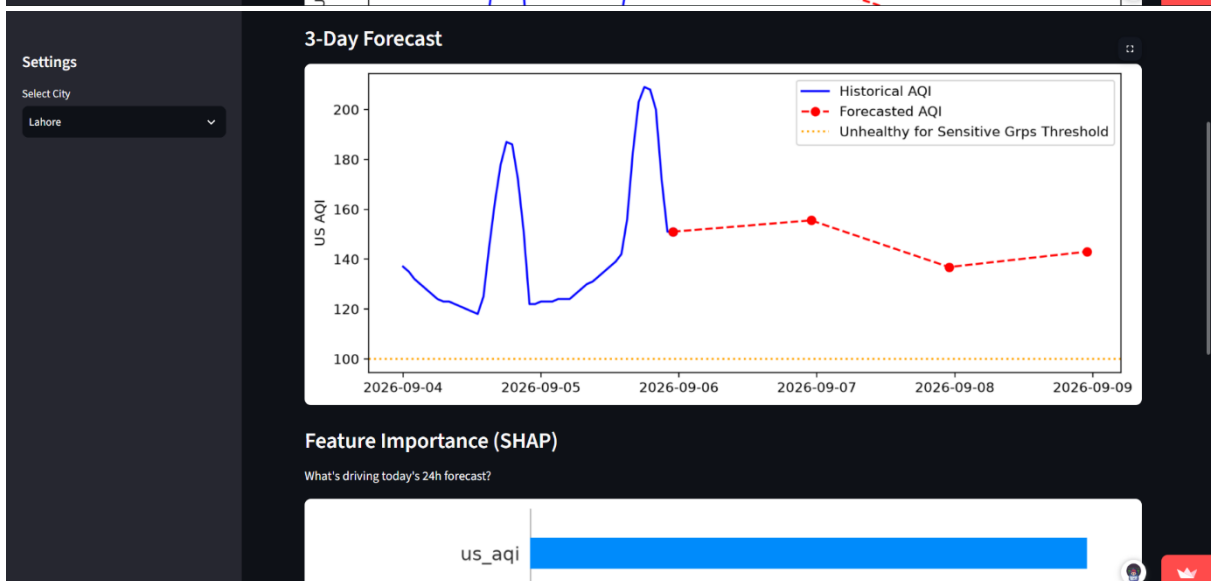
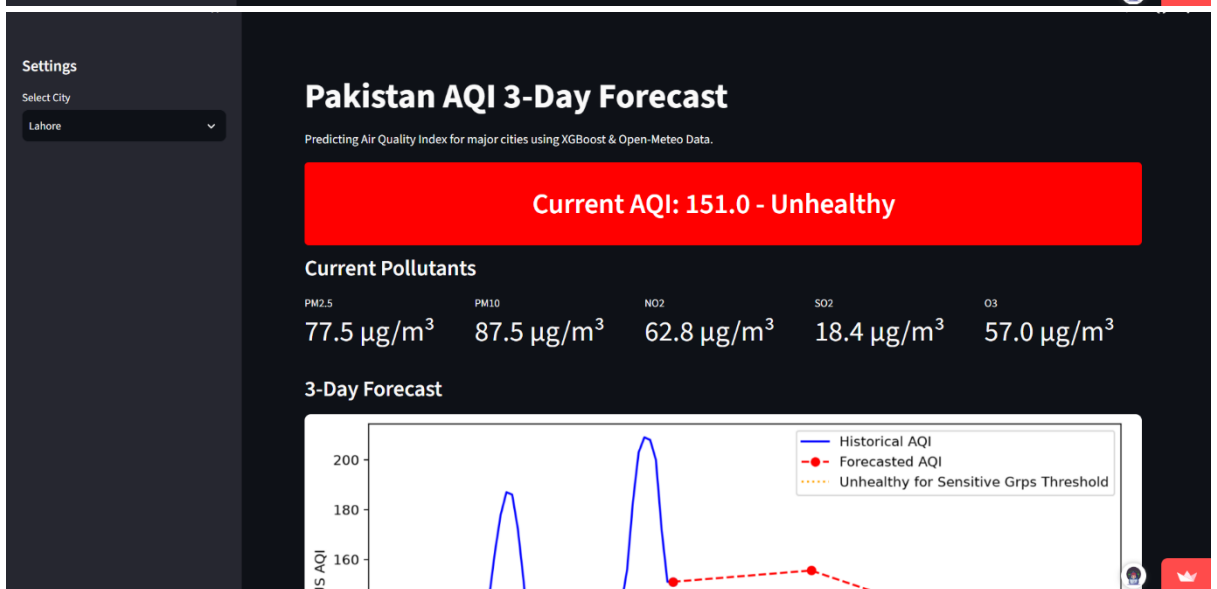
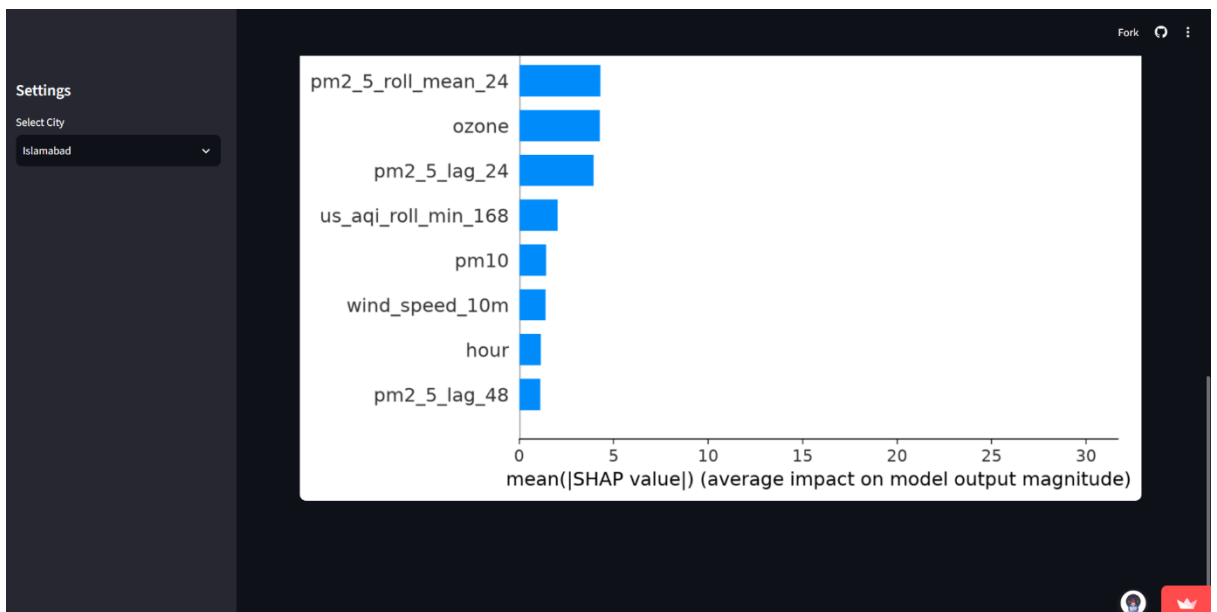
These challenges reflect the difference between building a model in a notebook versus building a working, automated, cloud-connected system most of the effort went into making each component reliable, not just functionally correct.

## 7. Dashboard

The Streamlit dashboard provides a simple, real-time interface for exploring the AQI forecasts:

- **City selector** – switch between Islamabad, Karachi, Hyderabad, and Lahore
- **Hazard banner** – a color-coded banner showing the current AQI value and its category (Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy, or Hazardous), based on standard US EPA breakpoints
- **Current pollutant levels** – live PM2.5, PM10, NO2, SO2, and O3 concentrations
- **3-day forecast chart** – shows the last 48 hours of historical AQI alongside the model's 24h/48h/72h forecast, with a reference line marking the "unhealthy" threshold
- **SHAP explainability panel** – a bar chart showing which features most influenced the current prediction, giving transparency into the model's reasoning rather than treating it as a black box.







## 8. Conclusion and Future Work

This project successfully delivers a complete, working AQI forecasting system: from raw data collection through to a deployed, publicly accessible dashboard. Beyond the machine learning itself, a significant part of the work involved solving infrastructure and deployment challenges network incompatibilities, dependency conflicts, and cloud platform constraints which are common in real production ML systems but rarely covered when learning ML in isolation.

Potential future improvements include:

- Automating the hourly/daily pipelines fully once the GitHub Actions network issue is resolved

- Adding a side-by-side multi-city comparison view
- Sending automated alerts (e.g., via Telegram) when AQI crosses unhealthy thresholds
- Adding anomaly detection for unusual pollution spikes
- Periodically retraining the model as more data accumulates